ARTFOLIO

Dossier de Inversión (< 10 Minutos) y Manual Técnico de Arquitectura y Ubicación de Archivos

Sección del Documento	Propósito en la Evaluación	Audiencia / Uso
PARTE I: Pitch de Inversión (< 10 min)	Presentar Artfolio paso a paso por categorías (Origen, Objetivo, Modelo Zero-Storage, Comparativa Competitiva y Demo Funcional).	Exposición directa al Profesor en rol de Cliente / Inversor.
PARTE II: Defensa de Proveedores y Whitelists	Explicación profunda de Whitelists, YouTube, Pinterest, DeviantArt y Google Docs, detallando cómo extraen imágenes/embeds y sus archivos.	Respaldo técnico ante preguntas sobre integraciones externas.
PARTE III: Anexo Q&A de Arquitectura Interna	Servicio de Mail/Confirmación y cómo acceder a él, Base de Datos, Acceso de Visitantes sin cuenta, API de Países y Seguridad.	Sección independiente para el interrogatorio técnico del evaluador.

PARTE I — EXPOSICIÓN EJECUTIVA CRONOMETRADA (DURACIÓN TOTAL: 9 MINUTOS)

Presentación Comercial y de Producto para el Cliente / Inversor

Guion estructurado en 5 categorías progresivas para presentar el valor comercial, financiero y funcional de Artfolio en menos de 10 minutos.

Categoría de la Presentación	Tiempo Estimado	Mensaje Central para el Inversor
Categoría 1: Origen, Problema y Objetivo	Min 0:00 – 1:45 (1m 45s)	Unificar el portafolio creativo multidisciplinario sin pagar costos millonarios de almacenamiento en la nube.
Categoría 2: Modelo 'Zero-Binary Storage'	Min 1:45 – 3:15 (1m 30s)	Márgenes operativos >95%: las CDNs gigantes pagan el ancho de banda; Artfolio aporta la curaduría e inteligencia.
Categoría 3: Comparativa Competitiva	Min 3:15 – 5:00 (1m 45s)	Superior a Linktree en inmersión visual y más versátil que ArtStation/Behance en soporte multi-formato y ética IA.
Categoría 4: Recorrido de Funciones Clave	Min 5:00 – 7:45 (2m 45s)	Visor 5-en-1, Filtro Anti-IA, Perfiles con Recorte de Avatar/Banner, Colecciones Privadas/No-Listadas y Dashboard KPI.
Categoría 5: Seguridad y Cierre de Inversión	Min 7:45 – 9:00 (1m 15s)	Plataforma autolimpiante (auditoría de enlaces, moderación, sesiones seguras) lista para escalar con riesgo mínimo.

Categoría 1: El Origen, el Problema de Mercado y el Objetivo (Min 0:00 – 1:45)

1. El Origen del Proyecto (El Dolor del Creador): Hoy en día, el creador digital moderno es multidisciplinario: un mismo profesional publica ilustraciones en Pinterest o DeviantArt, sube sus reels de animación a YouTube, aloja modelos 3D interactivos en Sketchfab, compone bandas sonoras en Spotify y redacta guiones o casos de estudio en Google Docs.

2. El Problema Financiero para el Inversor: Cuando un reclutador o estudio quiere evaluar a ese artista, recibe entre 5 y 7 enlaces dispersos. El reclutador debe saltar entre pestañas con interfaces distintas, anuncios y distracciones, perdiendo más del 40% de conversión en contratación. Por otro lado, desde la perspectiva de un inversor tecnológico, crear una red social tradicional donde los usuarios suban archivos pesados (videos 4K, mallas 3D, audio sin compresión) destruye la rentabilidad inicial debido a las facturas exponenciales de almacenamiento en Amazon S3 y transferencia CDN.

3. El Objetivo de Artfolio: Artfolio nace para resolver ambos problemas simultáneamente: es un Hub de Curaduría Multimedia Universal que unifica imágenes, video, 3D, audio y documentos interactivos dentro de una sola galería profesional con estética de cine, operando bajo una arquitectura 'Zero-Binary Storage' (Cero Almacenamiento Binario Pesado).

> 💡 **FRASE DE APERTURA PARA EL CLIENTE / INVERSOR:** «Artfolio no compite contra YouTube, Pinterest o Sketchfab alojando archivos pesados; los convierte en nuestros proveedores gratuitos de infraestructura CDN mientras nosotros somos dueños de la identidad profesional, la curaduría y el descubrimiento del talento.»

Categoría 2: Innovación Financiera — Arquitectura 'Zero-Binary Storage' (Min 1:45 – 3:15)

- **Delegación Inteligente de Ancho de Banda:** En lugar de almacenar gigabytes de video o texturas en servidores propios, el motor de Artfolio recibe únicamente el enlace original de la obra, verifica su autenticidad y extrae sus metadatos oficiales mediante protocolos oEmbed y OpenGraph.
- **Escalabilidad Financiera Inmediata:** Almacenar 100.000 obras multimedia en Artfolio ocupa menos de 150 MB en nuestra base de datos relacional MySQL. Una plataforma tradicional requeriría varios Terabytes en S3 y miles de dólares mensuales en tráfico de red.
- **Fricción Cero en Adopción de Usuarios:** El artista no tiene que volver a renderizar, comprimir ni resubir sus obras; pega el enlace de su plataforma habitual y en menos de 1.5 segundos Artfolio genera su tarjeta visual e integra el reproductor nativo.

Categoría 3: Comparativa con Plataformas Similares (Min 3:15 – 5:00)

Si el inversor pregunta por qué un artista elegiría Artfolio frente a alternativas existentes, esta matriz demuestra nuestro posicionamiento único en el mercado:

Criterio / Capacidad	ArtStation / Behance	DeviantArt / Pinterest	Linktree / Beacons	ARTFOLIO (Nuestra Propuesta)
Soporte Nativo Multi-Formato (5 en 1)	Limitado (Enfocado en imagen/video propio; débil en audio/docs).	No (Enfocados casi exclusivamente en imagen 2D).	No (Solo muestra botones de texto que sacan al usuario del sitio).	SÍ: Imagen HD, Video, Audio (Spotify), 3D (Sketchfab) y Docs en un solo visor.
Costo de Infraestructura	Muy Alto (Alojamiento	Muy Alto (Millones de	Bajo (Solo enlaces	Ultra Bajo: Experiencia

por Usuario	binario pesado en nube propia).	imágenes alojadas en CDN propia).	planos sin reproductor multimedia).	rica tipo ArtStation con el costo ligero de Linktree.
Transparencia y Filtro de IA Generativa	Polémico / Configuración compleja y rechazada por artistas.	Saturado de contenido IA sin filtrado claro para clientes.	Inexistente (No clasifica obras creativas).	Nativo desde publicación: Sello 'Con IA' y filtro en 1 clic ('Sin IA' / 'Con IA').
Colecciones con Privacidad de 3 Niveles	Básico (Portafolio público o borradores privados).	Tableros públicos o secretos simples.	No aplica.	Pública, No-Listada (enlace exclusivo para clientes con token) y Privada.
Personalización Visual sin Subir Archivos	Requiere subir avatares y banners pesados al servidor.	Requiere subida de archivos locales.	Subida local limitada o plantillas de pago.	Editor interactivo de encuadre y zoom (X, Y, Zoom) para Avatar y Banner por URL.

Categoría 4: Demostración de Funciones por Módulos (Min 5:00 – 7:45)

Durante este bloque de la presentación, se muestra al cliente cómo interactúan los usuarios dentro de la aplicación dividida en 6 pilares funcionales:

- **1. Explorador Público Inteligente y Filtro Ético de IA:** Búsqueda en tiempo real con debounce de 400ms, filtrado por las 5 categorías (Imagen, Video, Audio, 3D, Docs), ordenamiento por popularidad (favoritos) o recencia, y selector de Transparencia de Inteligencia Artificial (Todas / Solo talento humano 'Sin IA' / 'Con IA').
- **2. Visor Multimedia Adaptativo 5-en-1:** Al hacer clic en cualquier obra, se abre un modal cinematográfico sin recargar la página que adapta su motor según el formato: renderiza imágenes en resolución original para Pinterest/DeviantArt, o despliega un reproductor interactivo 16:9 seguro para YouTube, Sketchfab, Spotify y Google Docs.
- **3. Curaduría en Colecciones (Públicas, No-Listadas y Privadas):** Cada tarjeta del explorador y el visor modal cuentan con un botón desplegable de Colecciones (para usuarios registrados). Si el usuario aún no tiene colecciones, el menú ofrece la acción instantánea 'Create collection' que crea la colección y guarda la obra en el mismo paso. Además, cada colección puede configurarse como Pública, No-Listada (accesible solo mediante enlace secreto para enviar un pitch privado a un cliente) o Privada.
- **4. Onboarding Guiado y Perfil Público con Encuadre de Avatar y Banner:** Cuando un artista se registra, un asistente de 3 pasos guía su configuración inicial (identidad artística, ubicación con selector inteligente de países con banderas, habilidades y disponibilidad laboral). Posteriormente, en su Centro de Configuración dispone de un editor interactivo de arrastrar y hacer zoom tanto para su Avatar circular como para su Banner rectangular panorámico, guardando coordenadas matemáticas (X, Y, Zoom) que se reflejan en la cabecera superior de su Public Profile.
- **5. Publicación con Declaración de IA y Etiquetado:** Al publicar una obra, el artista cuenta con un interruptor dedicado para declarar si la obra incluye trabajo asistido por Inteligencia Artificial ('Obra con IA'). Esto protege la confianza de los reclutadores que buscan habilidades manuales específicas.
- **6. Dashboard de Rendimiento para el Artista:** Panel ejecutivo de 3 tarjetas con indicadores clave (KPIs): Total de Obras Activas (con desglose por categoría), Impacto de Favoritos recibidos (con la obra estrella más destacada) y Estado de Visibilidad Profesional ('Disponible para contratos' vs 'Portafolio en exhibición') junto al ranking del Top 5 de obras más exitosas.

Categoría 5: Gobernanza, Moderación y Cierre de Inversión (Min 7:45 – 9:00)

- **Salud Automática del Catálogo (Self-Healing Catalog):** Un riesgo clásico de los portafolios basados en enlaces es que el enlace externo muera. Artfolio incluye un comando automatizable ('artworks:check-links') que verifica periódicamente el estado HTTP de las obras, oculta automáticamente del explorador los enlaces caídos (404/410) y envía un correo de alerta al artista para que lo actualice.
- **Moderación y Convivencia Comunitaria:** Sistema de denuncias con validación anti-duplicados, bloqueo bidireccional entre usuarios (invisibiliza obras mutuas en el explorador), papelera de reciclaje con ventana de recuperación de 30 días y suspensión administrativa inmediata para cuentas infractoras.

PARTE II — DEFENSA TÉCNICA DE PROVEEDORES MULTIMEDIA Y WHITELISTS

Funcionamiento Interno y Ubicación Exacta de Archivos (Whitelists, YouTube, Pinterest, DeviantArt y Google Docs)

Guía detallada para responder con precisión quirúrgica cuando el profesor pregunte cómo funcionan las integraciones externas y en qué archivos están programadas.

1. Sistema de Whitelists (Listas Blancas) y Protección Anti-SSRF

¿Por qué existe y qué problema resuelve?: Como el servidor backend de Artfolio realiza peticiones HTTP salientes para extraer miniaturas y metadatos de los enlaces que ingresan los usuarios, aceptar cualquier URL arbitraria expondría al servidor a ataques de SSRF (Server-Side Request Forgery), donde un atacante podría intentar escanear la red interna (ej. http://127.0.0.1 o puertos locales) o inyectar sitios maliciosos.

¿Cómo funciona paso a paso?: La validación actúa en tres pasos dentro del método processUrl(string $url) de EmbedService.php:

- **Filtro Anti-SSRF:** Paso 1 (Validación de Formato y Bloqueo de Red Local): Se analiza la URL con filter_var($url, FILTER_VALIDATE_URL) y parse_url(). Luego se bloquea explícitamente cualquier dirección IP cruda (filter_var($host, FILTER_VALIDATE_IP)) o el dominio 'localhost' (líneas 68–70), neutralizando ataques SSRF.
- **Limpieza de Dominio:** Paso 2 (Normalización del Host): Se convierte el host a minúsculas y se limpia el prefijo 'www.' mediante preg_replace('/^www\./', '', $host).
- **Verificación de Lista Blanca:** Paso 3 (Cotejo Estricto contra $allowedDomains): Se recorre el diccionario de proveedores autorizados (pinterest, deviantart, spotify, youtube, sketchfab, vimeo, soundcloud, googledocs, imgur). Solo se aprueba si el host coincide exactamente con uno de los dominios de la lista blanca o es un subdominio legítimo terminado en '.dominio' (str_ends_with($cleanHost, '.' . $domain)). Si no coincide con ninguno, lanza una excepción controlada y rechaza la publicación.

```
🗁 UBICACIÓN EXACTA DE ARCHIVOS — WHITELISTS
1) Archivo Principal (Motor de Whitelist de Obras):
   -> app/Services/EmbedService.php
      - Propiedad protegida $allowedDomains (Líneas 12 a 32): Define los dominios permitidos por proveedor.
      - Método processUrl(string $url) (Líneas 41 a 90): Ejecuta el bloqueo anti-IP/localhost y valida la Whitelist.

2) Validación de Entrada en el Formulario de Creación/Edición de Obra:
   -> app/Http/Requests/ArtworkRequest.php (Líneas 22 a 33)
      - Valida 'source_url' => ['required', 'url', 'max:500'] antes de llegar al controlador.

3) Controlador que Invoca la Whitelist y Captura Errores:
   -> app/Http/Controllers/ArtworkController.php
      - Método previewUrl() (Líneas 27 a 53): Endpoint AJAX (/artworks/preview) que valida la URL en vivo.
      - Método store() (Líneas 58 a 95): Revalida la URL en backend antes de guardar en la base de datos.

4) Whitelist Adicional para Imágenes de Avatar y Banner de Perfil:
   -> app/Http/Requests/ProfileUpdateRequest.php (Líneas 32 a 85)
      - Valida que 'avatar_url' y 'banner_url' provengan de HTTPS y pasen por una lista blanca de CDNs de imagen seguras.
```

2. Implementación de Enlaces de Videos de YouTube

¿Cómo funciona la integración de YouTube?: YouTube permite múltiples formatos de enlace (enlaces largos de navegador watch?v=..., enlaces cortos móviles youtu.be/..., enlaces de YouTube Shorts shorts/... y enlaces /embed/...). Artfolio los unifica todos en un reproductor limpio y extrae la mejor miniatura disponible.

- **1. Extracción del ID de 11 caracteres:** En el método fetchYouTubeData(string $url, array &$result), se aplica la expresión regular: #(?:youtube\.com/(?:watch\?.*v=|shorts/|embed/)|youtu\.be/)([a-zA-Z0-9_-]{11})#i para capturar el identificador único de 11 caracteres ($videoId).
- **2. Consulta al API Oficial oEmbed:** Se realiza una petición GET con timeout de 5 segundos a https://www.youtube.com/oembed?url=https://www.youtube.com/watch?v={$videoId}&format=json para obtener el título oficial del video.
- **3. Verificación Inteligente de Miniatura HD (maxresdefault vs hqdefault):** No todos los videos de YouTube tienen miniatura HD (1280x720). Por eso, Artfolio hace una petición rápida Http::timeout(3)->head() a https://img.youtube.com/vi/{$videoId}/maxresdefault.jpg. Si responde HTTP 200 OK, usa maxresdefault.jpg; si da 404, degrada automáticamente a hqdefault.jpg (que siempre existe).
- **4. Generación del Iframe Seguro (Sandbox):** Construye la URL embebible https://www.youtube.com/embed/{$videoId}?rel=0 y genera un <iframe> con atributos de seguridad sandbox="allow-scripts allow-same-origin allow-presentation", asignando render_type = 'iframe' y category = 'video'.
- **5. Optimización de Rendimiento en Grilla:** En el modelo Artwork.php (método getPreviewUrlAttribute), cuando la obra se muestra en las tarjetas pequeñas del explorador, reemplaza dinámicamente maxresdefault.jpg por hqdefault.jpg para ahorrar ancho de banda y acelerar la carga de la grilla.

```
🗁 UBICACIÓN EXACTA DE ARCHIVOS — YOUTUBE
1) Extracción de ID, oEmbed, Miniatura y Construcción del Iframe:
   -> app/Services/EmbedService.php
      - Método fetchYouTubeData(string $url, array &$result) (Líneas 385 a 444).

2) Optimización Dinámica de Miniatura para el Explorador:
   -> app/Models/Artwork.php
      - Accessor getPreviewUrlAttribute() (Líneas 44 a 57): Cambia maxresdefault por hqdefault en
tarjetas.

3) Renderizado Visual en la Interfaz (Frontend Blade):
   -> resources/views/components/artwork-card.blade.php (Líneas 36 a 62): Muestra la miniatura y el
badge 'VIDEO'.
   -> resources/views/components/viewer-modal.blade.php (Líneas 120 a 136): Inyecta el <iframe> 16:9
interactivo.
```

3. Implementación de Enlaces de Pinterest y Extracción de Imagen HD

¿Por qué Pinterest requiere una ingeniería especial y cómo toma la imagen?: Pinterest es uno de los proveedores técnicamente más desafiantes porque: (a) los usuarios suelen compartir enlaces acortados de 'pin.it', (b) su API oEmbed entrega por defecto miniaturas pequeñas de 236 píxeles (/236x/), y (c) su servidor de imágenes (i.pinimg.com) bloquea la carga si detecta que otra web intenta incrustar la imagen (Hotlink Protection). Artfolio resuelve los tres obstáculos:

- **Paso 1 — Resolución de Redirecciones Acortadas (pin.it):** Si el usuario pega un enlace corto (ej. https://pin.it/...), fetchPinterestData() realiza primero una petición HTTP siguiendo redirecciones y extrae $res->effectiveUri() para obtener la URL canónica real del Pin (líneas 153–163 de EmbedService.php).
- **Paso 2 — Escalado Automático de Baja Resolución (/236x/) a Original (/originals/):** Consulta https://www.pinterest.com/oembed.json?url=... que devuelve un JSON con 'thumbnail_url' (normalmente en baja

resolución, ej. https://i.pinimg.com/236x/...). Nuestro algoritmo intercepta esa ruta y mediante preg_replace('#/(236x|474x|564x|736x)/#', '/originals/', $thumb) la transforma a la versión original sin compresión (/originals/). A continuación, dispara una petición HEAD de 2 segundos para verificar que el archivo original exista en el CDN de Pinterest; si no existe, hace fallback automático a alta definición (/736x/).

- **Paso 3 — Doble Motor de Respaldo (JSON-LD y OpenGraph):** Si un Pin específico no responde en el endpoint oEmbed, el método descarga el HTML del Pin y busca primero bloques estructurados <script type="application/ld+json"> y, en última instancia, la etiqueta <meta property="og:image">, aplicando igualmente la promoción a /originals/ o /736x/.
- **Paso 4 — Doble Resolución Dinámica (Grilla vs Modal):** En Artwork.php, el accessor preview_url transforma dinámicamente la URL a /474x/ para que el Explorador cargue rápido, mientras que high_res_url fuerza /originals/ cuando el usuario abre la obra en pantalla completa en el Visor Modal.
- **Paso 5 — Bypass Legítimo de Bloqueo de Hotlinking:** Para evitar que el CDN i.pinimg.com bloquee la imagen por venir desde nuestro dominio local, las etiquetas <img> en Blade incluyen el atributo referrerpolicy="no-referrer", logrando que la imagen cargue limpia como un elemento nativo (render_type = 'image').

```
🗁 UBICACIÓN EXACTA DE ARCHIVOS — PINTEREST
1) Motor de Extracción de Imagen de Pinterest (Redirección pin.it + oEmbed + Escalado /originals/ +
Fallback OG):
   -> app/Services/EmbedService.php
      - Método fetchPinterestData(string $url, array &$result) (Líneas 147 a 250).

2) Conversión Dinámica de Resolución (/474x/ en Tarjetas vs /originals/ en Visor Modal):
   -> app/Models/Artwork.php
      - Accessor getPreviewUrlAttribute() (Líneas 44 a 49): Convierte a /474x/ en la grilla.
      - Accessor getHighResUrlAttribute() (Líneas 62 a 68): Convierte a /originals/ en el modal.

3) Renderizado con Protección Anti-Hotlink (referrerpolicy="no-referrer"):
   -> resources/views/components/artwork-card.blade.php (Línea 40).
   -> resources/views/components/viewer-modal.blade.php (Líneas 139 y 146).
```

4. Implementación de Enlaces de DeviantArt

¿Cómo toma la imagen y los datos de DeviantArt?: DeviantArt aloja ilustraciones digitales de alta resolución en dominios como deviantart.com y enlaces cortos sta.sh. Su integración se resuelve mediante su backend oficial de oEmbed con respaldo OpenGraph:

- **1. Consulta al Backend Oficial oEmbed de DeviantArt:** El método fetchDeviantArtData() realiza una petición GET a https://backend.deviantart.com/oembed?url={urlencode($url)} enviando un cabecera User-Agent estándar.
- **2. Captura de Imagen en Resolución Completa:** A diferencia de otros servicios que solo entregan una miniatura pequeña, el JSON de DeviantArt devuelve en el campo $data['url'] la ruta directa al archivo gráfico completo en sus servidores (y $data['thumbnail_url'] como respaldo). Artfolio prioriza $data['url'] para garantizar nitidez total y establece render_type = 'image'.
- **3. Atribución Automática de Autoría:** Extrae $data['title'] y, si está presente $data['author_name'], formatea automáticamente el título sugerido como «Título de la Obra - por Autor».
- **4. Respaldo por Scraping de Metaetiquetas OpenGraph:** Si por políticas de privacidad de una desviación el endpoint oEmbed no devuelve la imagen, el código ejecuta un segundo intento leyendo directamente el HTML de la página y extrayendo mediante expresiones regulares <meta property="og:image"> y <meta property="og:title">.

```
🗁 UBICACIÓN EXACTA DE ARCHIVOS — DEVIANTART
1) Extracción oEmbed y Fallback OpenGraph de DeviantArt:
```

```
   -> app/Services/EmbedService.php
      -Lista blanca: 'deviantart' => ['deviantart.com', 'sta.sh'] (Línea 14).
      - Método fetchDeviantArtData(string $url, array &$result) (Líneas 256 a 315).

2) Visualización como Imagen Nativa de Alta Resolución:
   -> resources/views/components/artwork-card.blade.php (Líneas 38 a 45).
   -> resources/views/components/viewer-modal.blade.php (Líneas 137 a 148).
```

5. Implementación de Enlaces de Google Docs y Google Drive

¿Cómo funciona la transformación de enlaces de Google Docs?: Para que escritores, guionistas, diseñadores de narrativa (Game Designers) y directores de arte puedan exhibir guiones, presentaciones o hojas de cálculo sin obligar al visitante a descargar archivos PDF/Word, Artfolio integra Google Docs y Google Drive en vivo:

- **1. Detección de Tipo de Documento y Extracción de ID:** Los usuarios suelen copiar enlaces terminados en /edit, /view o ?usp=sharing (que no pueden incrustarse directamente en un iframe o muestran barras de edición pesadas). El método fetchGoogleDocsData() aplica la expresión regular #(?:docs|drive)\.google\.com/(document|presentation|spreadsheets|file)/d/([a-zA-Z0-9_-]+)#i para extraer dos datos clave: el tipo de documento ($docType) y el ID alfanumérico del archivo ($docId).
- **2. Conversión Automática al Visor Limpio (/preview):** Si es un archivo general de Drive ($docType === 'file'), construye https://drive.google.com/file/d/{$docId}/preview. Si es un documento, presentación u hoja de cálculo, construye https://docs.google.com/{$docType}/d/{$docId}/preview. El endpoint /preview de Google genera un lector limpio, rápido y de solo lectura ideal para portafolios.
- **3. Generación de Miniatura Visual sin API Key:** Google Docs no posee un endpoint oEmbed público. Para que la tarjeta en el explorador tenga una vista previa real de la primera página del documento sin requerir una API Key privada, Artfolio utiliza el generador oficial de miniaturas de Drive: https://drive.google.com/thumbnail?id={$docId}&sz=w800.
- **4. Extracción del Título Real y Renderizado Seguro:** Realiza una petición GET a la URL para leer la etiqueta <meta property="og:title"> o <title>, limpia los sufijos '- Google Docs' / '- Google Drive', asigna category = 'docs' y render_type = 'iframe', e inyecta un <iframe> con sandbox="allow-scripts allow-same-origin allow-popups".

```
🗁 UBICACIÓN EXACTA DE ARCHIVOS — GOOGLE DOCS Y DRIVE
1) Transformación a /preview, Miniatura de Drive y Scraping de Título:
   -> app/Services/EmbedService.php
      - Lista blanca: 'googledocs' => ['docs.google.com', 'drive.google.com'] (Línea 29).
      - Método fetchGoogleDocsData(string $url, array &$result) (Líneas 490 a 560).

2) Renderizado en Tarjeta (con fallback visual de icono de documento si el archivo restringe miniatura)
y Visor:
   -> resources/views/components/artwork-card.blade.php (Líneas 38 a 62).
   -> resources/views/components/viewer-modal.blade.php (Líneas 120 a 136).
```

Nota Complementaria: Sketchfab (3D) y Spotify (Audio)

Cobertura Completa de las 5 Categorías: En el mismo archivo app/Services/EmbedService.php también se encuentran implementados los motores para las dos categorías restantes: fetchSketchfabData() (líneas 321–379, que consulta https://sketchfab.com/oembed para modelos 3D interactivos con WebGL) y fetchSpotifyData() (líneas 450–485, que consulta https://open.spotify.com/oembed para pistas, álbumes y listas de reproducción de audio).

PARTE III — ANEXO INDEPENDIENTE DE INTERROGATORIO TÉCNICO (Q&A DEL PROFESOR)

Servicio de Mail, Base de Datos, Acceso de Visitantes, API de Países y Seguridad

Respuestas estructuradas (aparte de la presentación al inversor) con explicaciones de flujo y rutas exactas de archivos para responder al profesor.

Pregunta 1: ¿Cómo funciona el Servicio de Mail y la Confirmación de Cuenta, cómo se accede a él y qué archivos lo manejan?

A. ¿Qué funciones cumple el servicio de correo?: Artfolio implementa un sistema completo de correo transaccional sobre Laravel 12 y Laravel Breeze, cubriendo 4 flujos críticos: (1) Confirmación/Verificación de correo al registrarse, (2) Recuperación de contraseña olvidada, (3) Formulario de contacto directo Cliente -> Artista en el perfil público, y (4) Alertas automáticas al artista cuando uno de sus enlaces externos se cae.

B. ¿Cómo funciona exactamente la Confirmación de Cuenta?: El modelo User implementa la interfaz MustVerifyEmail (en app/Models/User.php, línea 16). Cuando un usuario completa el formulario de registro en RegisteredUserController@store, Laravel dispara el evento Registered($user), que genera automáticamente un enlace firmado criptográficamente con hash SHA-1 y fecha de expiración (/verify-email/{id}/{hash}). Hasta que el usuario hace clic en ese enlace (procesado por VerifyEmailController), cualquier intento de entrar a rutas protegidas con el middleware 'verified' (como el Dashboard, Onboarding o Publicación de Obras) lo redirige a la pantalla de aviso 'Verifica tu dirección de correo electrónico' (verify-email.blade.php), desde donde puede reenviar el correo con límite anti-spam (throttle:6,1 = máximo 6 intentos por minuto).

C. ¿Cómo se configura y cómo se accede a la bandeja de correos durante la demostración?: En entorno de desarrollo y evaluación local (Laragon), el sistema está configurado en el archivo .env mediante el protocolo SMTP apuntando al servidor local de captura de correos Mailpit (o alternativamente al driver 'log'):

- **1. Acceso Visual en Vivo vía Mailpit Web UI (Recomendado para la Defensa):** En el archivo .env se define MAIL_MAILER=smtp, MAIL_HOST=127.0.0.1 y MAIL_PORT=1025. Laragon incluye Mailpit escuchando en el puerto 1025 (SMTP) y ofrece una interfaz web completa en http://127.0.0.1:8025 (o http://localhost:8025). Al abrir http://127.0.0.1:8025 en el navegador durante la defensa, el profesor verá llegar en tiempo real el correo de confirmación, podrá inspeccionar su diseño HTML oscuro y hacer clic en el botón 'Confirmar Cuenta' para activar el usuario al instante.
- **2. Comando de Diagnóstico Rápido por Consola:** Creamos un comando artesanal propio en routes/console.php (líneas 29–46). Ejecutando en la terminal: php artisan mail:test profesor@ejemplo.com, el sistema envía un correo de prueba inmediato y confirma por consola si la conexión con el servidor de correo está activa.
- **3. Modo Respaldo sin Servidor SMTP (Driver Log):** Si en .env se cambia MAIL_MAILER=log, todos los correos enviados (incluyendo el enlace firmado de confirmación) quedan impresos íntegramente en el archivo storage/logs/laravel.log.

```
🗁 UBICACIÓN EXACTA DE ARCHIVOS — SERVICIO DE MAIL Y CONFIRMACIÓN
1) Activación del Contrato de Verificación Obligatoria:
   -> app/Models/User.php (Línea 16): class User extends Authenticatable implements MustVerifyEmail

2) Controladores de Verificación y Registro:
   -> app/Http/Controllers/Auth/RegisteredUserController.php: Registra al usuario y dispara el correo.
   -> app/Http/Controllers/Auth/EmailVerificationPromptController.php: Muestra la vista de espera de
```

```
confirmación.
   -> app/Http/Controllers/Auth/VerifyEmailController.php: Valida el hash firmado y marca
email_verified_at.
   -> app/Http/Controllers/Auth/EmailVerificationNotificationController.php: Reenvía el enlace de
confirmación.

3) Rutas de Autenticación y Confirmación:
   -> routes/auth.php (Líneas 40 a 55): Define verify-email, verify-email/{id}/{hash} (middleware
'signed') y verification.send.

4) Clases Mailable Personalizadas del Proyecto y sus Plantillas HTML:
   -> app/Mail/ContactArtist.php  --> Vista: resources/views/emails/contact-artist.blade.php
   -> app/Mail/LinkDownAlert.php  --> Vista: resources/views/emails/link-down-alert.blade.php
   -> Vista de bloqueo hasta confirmar: resources/views/auth/verify-email.blade.php

5) Configuración y Comando de Prueba:
   -> .env y config/mail.php (Configuración de MAIL_MAILER, MAIL_HOST=127.0.0.1, MAIL_PORT=1025).
   -> routes/console.php (Líneas 29 a 46): Comando 'php artisan mail:test {email?}'.
```

Pregunta 2: ¿Dónde y cómo se almacenan los datos de los usuarios y sus obras?

A. Motor de Base de Datos y Filosofía de Almacenamiento: Todos los datos se almacenan en una base de datos relacional MySQL llamada 'artfolio' (gestionada a través del ORM Eloquent de Laravel 12). Siguiendo nuestra arquitectura Zero-Binary Storage, la base de datos no guarda archivos binarios pesados (ni blobs de video ni imágenes subidas al disco local), sino referencias normalizadas, metadatos verificados y vectores matemáticos de posicionamiento.

- **B. Tabla 'users' (Datos de Usuarios y Artistas):** Almacena credenciales seguras (password hasheado con Bcrypt), identidad pública (name, artistic_name, slug único para la URL /artista/{slug}, email, email_verified_at, role ['artist'|'admin'], status ['active'|'suspended']), perfil profesional (bio, location, work_status, y 3 columnas tipo JSON llamadas skills, software y hiring_types que Eloquent convierte automáticamente en arrays de PHP), e información visual no destructiva: avatar_url junto a sus coordenadas de recorte circular (crop_x, crop_y, crop_zoom) y banner_url junto a sus coordenadas de encuadre rectangular panorámico (banner_crop_x, banner_crop_y, banner_crop_zoom).
- **C. Tabla 'artworks' (Datos de las Obras):** Relacionada con 'users' mediante clave foránea user_id (con borrado en cascada onDelete('cascade')). Guarda: title, description, category ('imagen', 'video', 'audio', '3d', 'docs'), has_ai (booleano que indica si tiene trabajo de IA), provider (ej. 'youtube', 'pinterest', 'deviantart', 'googledocs'), render_type ('image' o 'iframe'), source_url (enlace original), image_url (miniatura extraída), embed_url (URL para el iframe), tags (texto separado por comas), status ('published', 'draft', 'archived'), link_status ('active', 'down') y deleted_at (para la papelera SoftDeletes de 30 días).
- **D. Tablas Relacionales Complementarias:** 1) Tabla 'bookmarks': Relación muchos-a-muchos de Favoritos (user_id + artwork_id con índice único compuesto para impedir dar like dos veces).
  2) Tabla 'collections' y tabla pivote 'artwork_collection': Almacena las colecciones creadas por usuarios, incluyendo su nivel de privacidad visibility ('public', 'unlisted', 'private') y su token criptográfico share_token para enlaces no-listados.
  3) Tabla 'reports': Guarda los reportes de moderación (user_id, artwork_id, reason, details, status).
  4) Tabla 'user_blocks': Almacena bloqueos entre usuarios (blocker_id, blocked_id).

```
🗁 UBICACIÓN EXACTA DE ARCHIVOS — BASE DE DATOS Y MODELOS
1) Modelos Eloquent (Representación en Código de las Tablas y sus Relaciones):
   -> app/Models/User.php: Define $fillable, $casts (skills/software/hiring_types como 'array') y
relaciones.
   -> app/Models/Artwork.php: Usa el trait SoftDeletes, define $appends = ['preview_url',
```

```
'high_res_url'] y relaciones.
   -> app/Models/Collection.php: Gestiona visibilidad ('public', 'unlisted', 'private') y generación de
share_token.
   -> app/Models/Bookmark.php y app/Models/Report.php.

2) Migraciones SQL (Estructura Exacta de las Tablas):
   -> database/migrations/0001_01_01_000000_create_users_table.php
   -> database/migrations/2026_02_25_001053_add_artfolio_fields_to_users_table.php
   -> database/migrations/2026_02_25_001054_create_artworks_table.php
   -> database/migrations/2026_02_25_001055_create_bookmarks_table.php
   -> database/migrations/2026_02_25_001056_create_collections_table.php
   -> database/migrations/2026_02_25_001057_create_reports_table.php
   -> database/migrations/2026_11_11_180000_add_p1_fields_to_users_and_artworks.php (Añade has_ai y
user_blocks).
   -> database/migrations/2026_11_11_185000_add_visibility_to_collections_table.php (Añade visibility y
share_token).
   -> database/migrations/2026_11_11_190500_add_banner_crop_fields_to_users_table.php (Añade
banner_crop_x/y/zoom).
```

Pregunta 3: ¿Cómo accede un usuario visitante sin tener cuenta y qué archivos manejan eso?

A. Filosofía de Navegación para Visitantes (Guests): Para maximizar la retención y permitir que clientes o reclutadores evalúen portafolios sin obligarlos a registrarse de entrada, Artfolio implementa una arquitectura de 'Acceso Público Abierto con Intercepción Progresiva de Acciones':

- **1. Qué puede hacer el visitante libremente (Rutas Públicas):** En routes/web.php, la ruta raíz GET / (Explorador), la ruta GET /artista/{slug} (Perfil Público del Artista), la ruta GET /collections/{collection} (Colecciones públicas o con enlace no-listado) y el formulario POST /artista/{slug}/contact (Contacto directo al artista) se encuentran FUERA del grupo de middleware 'auth'. Cualquier visitante anónimo puede buscar obras, filtrar por categorías o IA, abrir el visor modal interactivo, reproducir videos/modelos 3D y ver el perfil y banner del artista.
- **2. Cómo los Controladores distinguen entre Visitante y Usuario Registrado:** En los controladores ExplorerController y PublicProfileController se consulta Auth::check() y Auth::id(). Si el usuario es un visitante anónimo, las consultas omiten el cruce de bloqueos personales o colecciones privadas sin lanzar ningún error de sesión nula.
- **3. Cómo se interceptan las acciones exclusivas de usuarios registrados:** En las vistas Blade (artwork-card.blade.php y viewer-modal.blade.php), los botones de interacción social están envueltos en directivas condicionales @auth ... @else ... @endauth:
  - El selector desplegable de añadir a Colección (<x-collection-picker />) está dentro de @auth, por lo que solo se renderiza para usuarios con cuenta activa.
  - Cuando un visitante sin cuenta hace clic en 'Favorito' o 'Reportar', en lugar de enviar una petición fallida al servidor, el botón ejecuta en el navegador el evento Alpine.js: $dispatch('open-guest-modal', { context: 'favorito' }). Esto despliega al instante un modal persuasivo (guest-modal.blade.php) que le explica la función y le ofrece los botones 'Crear cuenta gratis' o 'Ya tengo una cuenta'.

```
🗁 UBICACIÓN EXACTA DE ARCHIVOS — ACCESO DE VISITANTES SIN CUENTA
1) Definición de Rutas Públicas (Sin Middleware 'auth'):
   -> routes/web.php
      - Línea 60: Route::get('/', [ExplorerController::class, 'index'])->name('home');
      - Línea 61: Route::get('/artista/{slug}', [PublicProfileController::class, 'show'])-
>name('artist.portfolio');
      - Línea 62: Route::post('/artista/{slug}/contact', [PublicProfileController::class, 'contact']);
      - Línea 115: Route::get('/collections/{collection}', [CollectionController::class, 'show']);
```

```
2) Controladores que Soportan Navegación Anónima:
   -> app/Http/Controllers/ExplorerController.php (Líneas 13 a 89).
   -> app/Http/Controllers/PublicProfileController.php (Líneas 16 a 63).

3) Vistas y Componente Interceptor de Visitantes (Guest Modal):
   -> resources/views/components/guest-modal.blade.php (Modal reactivo que escucha '@open-guest-
modal.window').
   -> resources/views/components/artwork-card.blade.php (Líneas 75 a 112: directivas @auth / @else).
   -> resources/views/components/viewer-modal.blade.php (Líneas 36 a 108: directivas @auth / @else).
   -> resources/views/layouts/navigation.blade.php (Muestra 'Iniciar Sesión / Registrarse' si es
visitante).
```

Pregunta 4: ¿De dónde se sacan los países disponibles en el registro/onboarding y perfil, y qué archivo los maneja?

A. Arquitectura de Triple Respaldo para los Países: Para que el selector de país con banderas funcione siempre de forma instantánea y nunca falle (incluso si durante la evaluación en clase no hubiera conexión a Internet o fallara una API externa), diseñamos un micro-servicio interno en Laravel con Caché de 30 días y Triple Fallback (3 niveles de respaldo):

- **1. Endpoint Interno con Caché de 30 Días (/api/countries):** En routes/web.php (líneas 15 a 57) está declarada la ruta GET /api/countries (nombre: api.countries). Utiliza Cache::remember('global_countries_list_v1', 86400 * 30, ...) para guardar el listado completo de países en caché durante 30 días, de modo que las consultas subsiguientes tardan menos de 2 milisegundos.
- **2. Fuente Primaria (Dataset Oficial GitHub mledoze/countries):** La primera vez que se solicita (si la caché está vacía), el backend hace una petición HTTP con timeout de 4 segundos al repositorio oficial de países en GitHub: https://raw.githubusercontent.com/mledoze/countries/master/countries.json, extrayendo los nombres comunes ($c['name']['common']) de más de 240 países y territorios.
- **3. Fuente Secundaria (API REST Countries v3.1):** Si GitHub no respondiera, el bloque catch consulta automáticamente la API pública https://restcountries.com/v3.1/all?fields=name.
- **4. Fuente Terciaria Offline (Arreglo Local en Código):** Si el servidor estuviera completamente desconectado de Internet, el endpoint devuelve un arreglo estático local de más de 100 países codificado directamente en las líneas 41 a 54 de routes/web.php, garantizando disponibilidad del 100%.
- **5. Consumo Reactivo en el Frontend:** Tanto en el Paso 1 del Onboarding inicial (step1.blade.php) como en la Pestaña de Perfil Público del Centro de Configuración (tab-profile.blade.php), un componente Alpine.js ejecuta fetch('/api/countries') al iniciar, hidrata el buscador desplegable y asocia un mapa de banderas Unicode junto a la opción especial 'Remoto / Global'.

```
🗁 UBICACIÓN EXACTA DE ARCHIVOS — ORIGEN Y GESTIÓN DE PAÍSES
1) Backend — Endpoint Proveedor de Países con Caché y Triple Fallback:
   -> routes/web.php (Líneas 15 a 57): Ruta GET '/api/countries' (->name('api.countries')).

2) Frontend — Selector Desplegable con Buscador y Banderas durante el Onboarding Post-Registro:
   -> resources/views/onboarding/step1.blade.php (Líneas 60 a 165 y script Alpine en líneas 215 a 265).

3) Frontend — Selector Desplegable de Países en Edición de Perfil:
   -> resources/views/profile/partials/tab-profile.blade.php (Líneas 195 a 280).

4) Validación y Guardado del País en la Base de Datos (Columna 'location' de la tabla 'users'):
   -> app/Http/Controllers/OnboardingController.php (Método saveStep1, líneas 35 a 58).
   -> app/Http/Requests/ProfileUpdateRequest.php (Línea 29: 'location' => ['nullable', 'string',
```

```
'max:120']).
```

Pregunta 5: Otras Funciones Clave que el Profesor Puede Cuestionar (Guía Rápida)

5.1. ¿Cómo funciona el Editor de Posicionamiento de Avatar (Circular) y Banner (Rectangular)?

Explicación Técnica: Tanto el Avatar como el Banner funcionan sin subir archivos al servidor: el usuario pega el enlace de su imagen y puede arrastrarla con el ratón (drag) y usar un deslizador de zoom (1x a 3x). En lugar de recortar físicamente un archivo nuevo, guardamos en la tabla 'users' los desplazamientos porcentuales y el factor de escala (crop_x, crop_y, crop_zoom para el Avatar; banner_crop_x, banner_crop_y, banner_crop_zoom para el Banner). Luego, cuando cualquier visitante entra al Public Profile del artista, aplicamos una transformación CSS acelerada por hardware: transform: translate(X%, Y%) scale(Zoom) dentro de un contenedor con overflow-hidden.

```
🗁 ARCHIVOS — ENCUADRE DE AVATAR Y BANNER
-> Editor Interactivo (Drag & Zoom): resources/views/profile/partials/tab-profile.blade.php (Líneas 18
a 175 y script en líneas 370 a 470).
-> Renderizado del Banner Superior y Avatar en Perfil Público: resources/views/profile/show.blade.php
(Líneas 10 a 68).
-> Validación de Coordenadas (-100 a 100 y Zoom 1 a 3): app/Http/Requests/ProfileUpdateRequest.php
(Líneas 86 a 91).
```

5.2. ¿Cómo funciona la Expiración de Sesión por Inactividad y el Aviso de Extensión?

Explicación Técnica: El tiempo de vida de la sesión está configurado en 10 minutos (SESSION_LIFETIME=10 en config/session.php). Para evitar que el artista pierda cambios mientras edita su portafolio, el componente session-timeout-modal.blade.php monitorea eventos de ratón, teclado y scroll. Si detecta 8 minutos de inactividad continua, muestra un modal con una cuenta regresiva de 120 segundos. Al pulsar 'Extender mi sesión', realiza un fetch POST a /session/keep-alive que regenera la marca de tiempo de sesión en el servidor sin recargar la página.

```
🗁 ARCHIVOS — SESIÓN E INACTIVIDAD
-> Configuración de Tiempo de Sesión: config/session.php (Línea 35).
-> Endpoint Keep-Alive: routes/web.php (Líneas 71 a 74: POST '/session/keep-alive').
-> Modal y Temporizador JS: resources/views/components/session-timeout-modal.blade.php.
```

5.3. ¿Cómo funcionan las Colecciones Públicas, No-Listadas y Privadas y el Botón Rápido en las Obras?

Explicación Técnica: Cada colección tiene una columna visibility ('public', 'unlisted', 'private') y un share_token aleatorio de 32 caracteres. La autorización se centraliza en CollectionPolicy.php: (a) 'public' es visible para todos y aparece en el perfil público del artista; (b) 'unlisted' se oculta del perfil público, pero si el dueño comparte el enlace con ?token=..., cualquier cliente con ese enlace puede verla; (c) 'private' bloquea el acceso incluso teniendo el enlace (solo el dueño puede abrirla). Además, cada obra incluye el componente <x-collection-picker /> que permite añadirla a una colección existente o crear una nueva en el acto vía AJAX (POST /collections/quick-create).

```
🗁 ARCHIVOS — COLECCIONES Y PRIVACIDAD
-> Política de Acceso (3 niveles de privacidad): app/Policies/CollectionPolicy.php.
-> Controlador (CRUD + Quick-Create AJAX): app/Http/Controllers/CollectionController.php.
-> Botón Desplegable en Obras: resources/views/components/collection-picker.blade.php.
```

```
-> Vista y Modal de Configuración de Privacidad: resources/views/collections/show.blade.php y
index.blade.php.
```

5.4. ¿Cómo funcionan la Papelera de 30 Días, el Bloqueo de Usuarios y la Suspensión de Cuentas?

- **Papelera de Reciclaje (30 días):** Cuando un artista elimina una obra, no se hace un DELETE físico inmediato; gracias al trait SoftDeletes en Artwork.php se llena la columna deleted_at y la obra pasa a la Papelera (TrashController.php), desde donde puede restaurarse o borrarse definitivamente. El comando programado 'php artisan artworks:purge-trash' (en routes/console.php) purga automáticamente las obras que llevan más de 30 días en la papelera.
- **Bloqueo Bidireccional:** Gestionado por UserBlockController.php y la tabla 'user_blocks'. Si el Usuario A bloquea al Usuario B, ExplorerController filtra las obras de ambos sentidos (whereNotIn('user_id', $excludedIds)) e impide visitar el perfil o enviar mensajes.
- **Suspensión Administrativa en Caliente:** El middleware EnsureUserIsActive.php verifica en cada petición si auth()->user()->status === 'suspended'. Si un administrador suspende una cuenta desde AdminController@toggleUserStatus, el middleware cierra la sesión del infractor inmediatamente y lo expulsa al login.

Mapa Maestro de Archivos del Proyecto (Referencia Rápida para el Examen)

Módulo / Función Preguntada	Archivo(s) Principal(es)	Método o Línea Clave
Whitelists y Anti-SSRF	app/Services/EmbedService.php	$allowedDomains (L.12-32), processUrl() (L.41-90)
Enlaces y Videos de YouTube	app/Services/EmbedService.php app/Models/Artwork.php	fetchYouTubeData() (L.385-444) getPreviewUrlAttribute() (L.44-57)
Enlaces e Imágenes de Pinterest	app/Services/EmbedService.php resources/views/components/artwork-card.blade.php	fetchPinterestData() (L.147-250) referrerpolicy="no-referrer" (L.40)
Enlaces de DeviantArt	app/Services/EmbedService.php	fetchDeviantArtData() (L.256-315)
Enlaces de Google Docs / Drive	app/Services/EmbedService.php	fetchGoogleDocsData() (L.490-560)
Servicio de Mail y Confirmación	app/Models/User.php routes/auth.php app/Mail/ContactArtist.php & LinkDownAlert.php	implements MustVerifyEmail (L.16) Rutas verify-email (L.40-55) Mailpit UI: http://127.0.0.1:8025
Almacenamiento (Usuarios y Obras)	app/Models/User.php & Artwork.php database/migrations/*.php	Tablas MySQL: users, artworks, collections, bookmarks, reports, user_blocks
Acceso de Visitante (Guest)	routes/web.php app/Http/Controllers/ExplorerController.php resources/views/components/guest-modal.blade.php	Rutas públicas (L.60-62, 115) Interceptor @open-guest-modal.window
Países Disponibles en Registro	routes/web.php resources/views/onboarding/step1.blade.php resources/views/profile/partials/tab-profile.blade.php	GET /api/countries con Caché 30d y 3 fallbacks (L.15-57)

Recorte de Avatar y Banner	resources/views/profile/partials/tab-profile.blade.php resources/views/profile/show.blade.php	Drag & Zoom interactivo -> crop_x/y/zoom y banner_crop_x/y/zoom
Colecciones y Privacidad	app/Policies/CollectionPolicy.php app/Http/Controllers/CollectionController.php resources/views/components/collection-picker.blade.php	Visibilidad public/unlisted/private y Quick-Create AJAX